Natural Language Processing | Machine Learning | REST API

IMDB Sentiment Analysis System

End-to-end NLP pipeline: 7-step preprocessing, TF-IDF + Logistic Regression, deployed via FastAPI REST API with responsive web frontend.

sentiment_analysis_nlp.ipynb | sentiment_api.py | index.html

PROJECT STATS

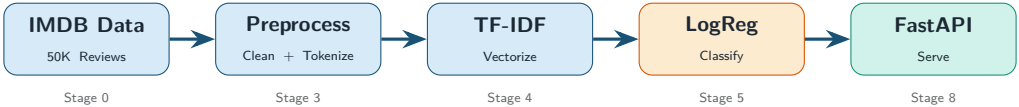
50,000
IMDB reviews

90.08%
Test accuracy

0.9649
ROC-AUC score

<10 ms
Inference / review

System Overview — *End-to-end pipeline from raw text to deployed REST API*



- 50,000 `.txt` IMDB reviews
- Balanced: 50 % pos / 50 % neg
- Parsed to `imdb_reviews.csv`
- 80 % train / 20 % test split

- 7-step NLP preprocessing
- TF-IDF with 10K features
- 5 models benchmarked
- LogReg wins: 90.08 %

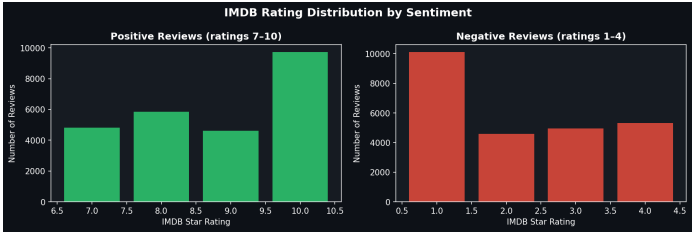
- FastAPI + Pydantic validation
- `/predict` and `/batch_predict`
- Swagger UI auto-docs
- CORS + HTML5 frontend

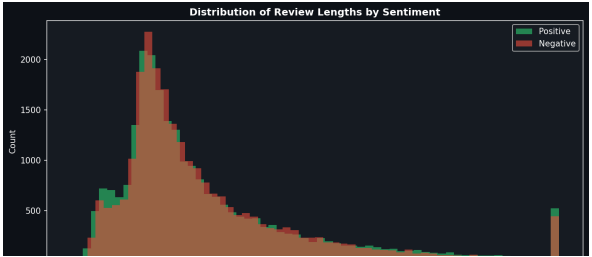
Dataset — IMDB Movie Reviews — *50,000 balanced reviews from Stanford AI Lab (aclImdb)*

Property	Value
Total reviews	50,000
Training set	40,000
Test set	10,000
Positive	25,000
Negative	25,000
Avg. length	230 words
Vocabulary	∼100K
TF-IDF feats	10,000

> *"This film is an absolute masterpiece. Superb acting!"*

> *"Terrible. A complete waste of two hours."*





NLP Preprocessing Pipeline — *7-step text cleaning before feature extraction*

1. **Lowercase** — "Movie" → "movie"
2. **HTML removal** — Strip <br>, entities
3. **Contractions** — "don't" → "do not"
4. **Remove punctuation** — Letters only
5. **Tokenization** — Split into words
6. **Stop-word removal** — Keep "not", "never"
7. **Lemmatization** — "loving" → "love"

Before: `<br>I DON'T think this is Good!!! WORST 2hrs`

After: `not think good worst hour`

```python
from bs4 import BeautifulSoup
from nltk.corpus import stopwords
from nltk.stem import
    WordNetLemmatizer

KEEP = {"not", "no", "never"}
stops = set(stopwords.words('english')
    )
stops -= KEEP
lem = WordNetLemmatizer()

def preprocess(text):
    text = BeautifulSoup(
        text, "html.parser").get_text
            ()
    text = re.sub(r"[^a-z\s]", "",
                  text.lower())
    return " ".join([
        lem.lemmatize(w)
        for w in text.split()
        if w not in stops
    ])
```

TF-IDF Feature Extraction — Top Predictive Words — *Logistic Regression coefficient weights reveal strongest sentiment signals*

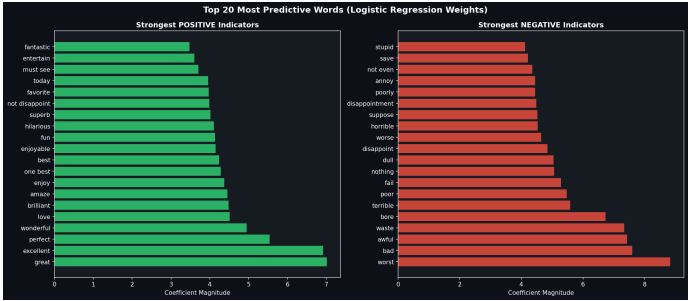
TF-IDF Formula

$$\text{score}(t, d) = \log(1+f_{t,d}) \cdot \log \frac{N+1}{df_t+1}$$

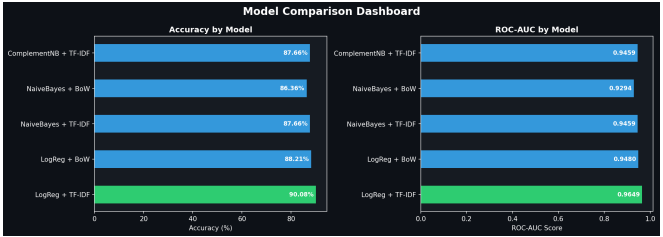
$f_{t,d}$: term count in doc; N: corpus size; df_t: doc freq.

Parameter	Value
`max_features`	10,000
`ngram_range`	(1, 2)
`min_df`	2
`max_df`	0.95
`sublinear_tf`	True
Matrix shape	40K × 10K

Bigrams capture phrases like *"must see"*, *"not disappoint"*, *"waste of time"*



Model Comparison — 5 Classifiers Benchmarked — *All models trained on identical TF-ID features; LogReg + TF-IDF wins*



Accuracy	90.08%
ROC-AUC	0.9649

Model	Acc.
NaiveBayes + BoW	86.36%
NaiveBayes + TF-IDF	87.66%
ComplementNB + TFIDF	87.66%
LogReg + BoW	88.21%
LogReg + TF-IDF	**90.08%**

TF-IDF consistently beats BoW by giving rare, informative words higher weight.

Scikit-learn Pipeline — Code — *TF-IDF + Logistic Regression chained in a single Pipeline obje*

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import\
    LogisticRegression
from sklearn.feature_extraction.text\
    import TfidfVectorizer
import joblib
# Build pipeline
pipeline = Pipeline([
  ('tfidf', TfidfVectorizer(
      max_features=10_000,
      ngram_range=(1, 2),
      sublinear_tf=True,
      min_df=2,
      max_df=0.95,
  )),
  ('clf', LogisticRegression(
      C=1.0,
      max_iter=1000,
      solver='lbfgs',
  )),
])
# Train & evaluate
```

Input — raw review string

↓

TF-IDF Vectorizer
10K sparse features, bigrams

↓

Logistic Regression
Weighted word coefficients

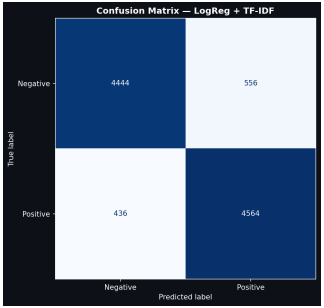
↓

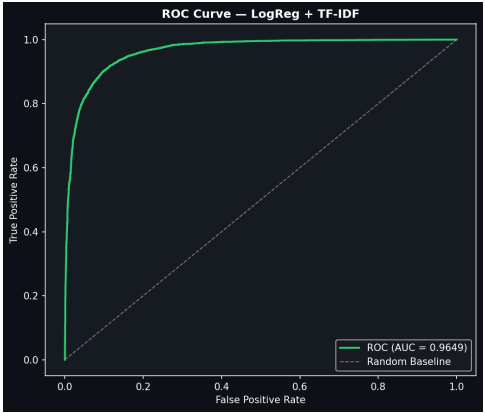
Output — pos / neg + probability

Why Pipeline? Wrapping vectorizer + model prevents data leakage during cross-validation and makes deployment trivial — one `joblib.dump()` call.

```
# Load & predict (in API)
model = joblib.load('
    sentiment_model.pkl')
pred  = model.predict(["Great film
```

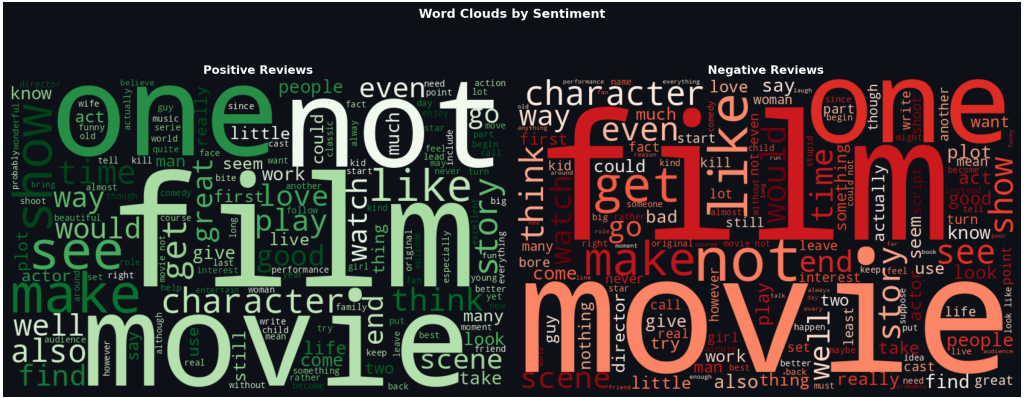
Model Evaluation & Performance — *LogReg + TF-IDF evaluated on the held-out 10,000-review test set*





Class	Prec	Rec	F1	N
Negative	0.889	0.888	0.889	5000
Positive	0.913	0.913	0.913	5000
Avg	**0.901**	**0.901**	**0.901**	10K

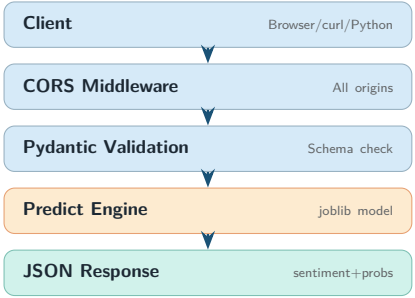
Exploratory Data Analysis — Word Clouds — *Most frequent words in positive vs. negative reviews after preprocessing*



Positive cloud: Words like *great*, *love*, *brilliant*, *enjoy*, *perfect* dominate — strong praise vocabulary.

Negative cloud: Words like *bad*, *worst*, *waste*, *bore*, *terrible* stand out — clear disappointment signals.

FastAPI Backend — sentiment_api.py — *Production-ready REST server with Pydantic validation and CORS*



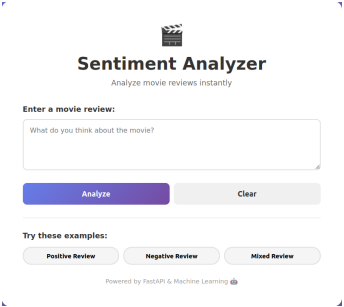
Method	Route	Use
GET	/	HTML UI
GET	/health	Status
POST	/predict	Single
POST	/batch_predict	Batch
GET	/docs	Swagger

```
// POST /predict  -- Request
{ "text": "This movie is fantastic!" }

// Response (200 OK)
{
  "text":          "This movie ...",
  "sentiment":     "positive",
  "confidence":    0.9453,
  "positive_prob":0.9453,
  "negative_prob":0.0547,
  "timestamp":     "2026-03-03T..."
}
```

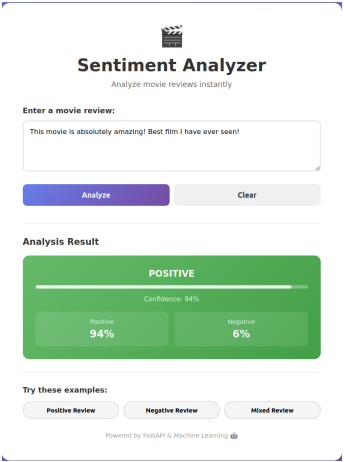
```
// POST /batch_predict
{ "texts": ["Great!", "Terrible."] }

// Response
{ "predictions": [{...}, {...}],
```

Web Frontend Demo — Live API Results — *HTML/CSS/JS frontend calling FastAPI — actual screenshots*

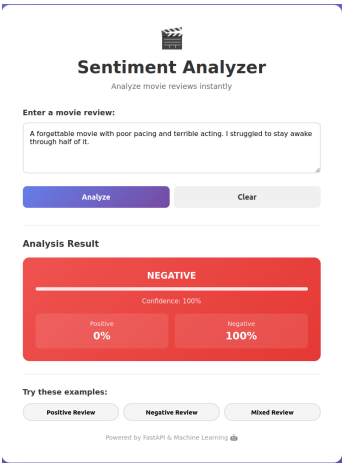
Default State



Positive Result



Negative Result



SUMMARY

What Was Built & What Comes Next

- ✓ 50K IMDB reviews with 7-step NLP pipeline
- ✓ TF-IDF (10K features, bigrams)
- ✓ 5 classifiers; LogReg+TF-IDF best at **90.08%**
- ✓ ROC-AUC = 0.9649
- ✓ FastAPI: `/predict`, `/batch_predict`
- ✓ Pydantic validation, CORS, Swagger docs
- ✓ Responsive HTML/JS frontend (live screenshots)

Future Improvements

- → Fine-tune **BERT** for +5% accuracy
- → Multilingual (mBERT / XLM-R)
- → Redis caching for repeated queries
- → Docker + Kubernetes deployment
- → CI/CD with GitHub Actions
- → Aspect-based sentiment analysis
- → Model drift monitoring

50K Reviews · TF-IDF · LogReg · 90.08% Acc · FastAPI · Live Frontend